

Construcción de una base de conocimiento vectorial

CODEFEST AD ASTRA 2026 — Etapa 1: Informe técnico

GEOTENSOR

Agosto de 2026

1. Resumen de la solución

El sistema construye una base de conocimiento vectorial única sobre los tres fenómenos del reto, a partir de un corpus heterogéneo que combina PDF, HTML, JSON, CSV, XLSX, Markdown, texto plano, imágenes y mapas OpenStreetMap en formato PBF. El flujo es lineal: cada archivo se clasifica por su tipo MIME real, se convierte a texto plano mediante la ruta de extracción propia de su formato, se fragmenta con un segmentador oracional diseñado específicamente para cumplir el requisito de completitud lingüística del reto, se codifica con `intfloat/multilingual-e5-small` bajo el esquema de prefijos asimétricos de la familia E5, y se inserta en un índice `IndexFlatIP` de FAISS con vectores normalizados, equivalente a búsqueda exacta por similitud coseno. En paralelo, cada fragmento deja un registro en un almacén de metadata en formato JSON Lines cuyo orden de líneas replica exactamente el orden de inserción en FAISS.

La decisión de diseño que articula todo el sistema es el segmentador oracional propio (clase `Chunkeador`). El reto prohíbe explícitamente que una oración cruce la frontera entre dos fragmentos, y ningún segmentador genérico disponible garantiza esa propiedad: todos terminan cortando por caracteres o por espacios cuando se quedan sin separadores. Por ello, la unidad atómica de construcción de los chunks no es el carácter sino la oración completa, detectada con un reconocedor de fin de oración que descarta abreviaturas, iniciales y numeraciones en tres idiomas.

Cuadro 1: Cifras generales de la base construida.

Elemento	Valor
Documentos indexados	TODO
Fragmentos (<i>chunks</i>)	TODO
Encoder	<code>intfloat/multilingual-e5-small</code> (MIT)
Dimensión del vector	384
Tipo de índice FAISS	<code>IndexFlatIP</code> sobre vectores normalizados
Límite por fragmento	400 tokens y 240 palabras
Solapamiento	1 oración completa entre chunks vecinos
Tamaño del índice en disco	TODO
Tiempo total de indexación	TODO
Grafo de conocimiento	No implementado (véase Sección 7)

2. Preprocesamiento y extracción de texto

2.1. Identificación del formato

El formato de cada archivo no se deduce de su extensión sino de su tipo MIME real, obtenido con `python-magic` a partir de los bytes de cabecera. Esta decisión responde a un problema concreto del corpus: las extensiones son poco fiables. Varios archivos `.xlsx` son reportados por el sistema como `application/zip` —que es lo que técnicamente son— y algunos `.csv` llegan como `text/plain`. El despachador contempla ambos casos con ramas explícitas que combinan el tipo MIME con la extensión, de modo que ningún archivo válido cae en la rama de formato no soportado por un desacuerdo entre ambas señales.

2.2. Tratamiento por formato

Cuadro 2: Estrategia de extracción por formato de origen.

Formato	Herramienta	Decisiones de extracción
PDF	pymupdf4llm	Conversión a Markdown preservando orden de lectura y jerarquía de encabezados, con OCR integrado en spa+eng+por para las páginas sin capa de texto. Las tablas sobreviven como bloques Markdown y se detectan luego por expresión regular.
HTML	BeautifulSoup HTMLHeaderTextSplitter	Se eliminan script y style ; las tablas se aplanan a texto celda celda ; las imágenes locales se sustituyen por su transcripción OCR en línea, de modo que las infografías no se pierden. La segmentación primaria usa la jerarquía h1–h4 .
JSON	json + aplanado recursivo	El objeto se aplanan a pares ruta.completa: valor , conservando la ruta de anidamiento como contexto del valor. Los campos vacíos o nulos se descartan.
CSV XLSX	/ pandas / openpyxl	Lectura por bloques de 1000 filas (read_only en Excel) para acotar la memoria. Cada fila se serializa como columna: valor columna: valor , y las columnas enteramente vacías se eliminan antes de serializar.
Imágenes	Pillow + pytesseract	OCR en spa+eng+por ; las imágenes sin texto legible se descartan en lugar de generar fragmentos vacíos.
PBF OSM	/ pyogrio + GeoPandas	Se enumeran las capas del mapa y se recorre cada una descartando la columna de geometría; los atributos restantes se serializan como texto y se prefijan con el nombre de la capa. Existe un respaldo de lectura tabular para archivos que la librería geoespacial no reconoce.
MD TXT	/ Lectura directa UTF-8	Se separan los bloques de tabla del texto corrido antes de fragmentar, para que cada uno siga su propia ruta.

2.3. Separación entre prosa y datos tabulares

Una observación transversal condicionó el diseño: el corpus mezcla, dentro de un mismo archivo, texto en prosa y estructuras tabulares. Aplicar reglas gramaticales a una tabla es contraproducente, porque una tabla no tiene oraciones y el segmentador oracional la trataría como una sola unidad gigantesca. Por eso, en PDF, Markdown y TXT se detectan los bloques de tabla mediante expresión regular y se enrutan a un método de fragmentación distinto (**split_datos**), que respeta los límites de tamaño pero no exige completitud oracional. El texto corrido va al segmentador oracional completo.

2.4. Identificación de documentos y fenómeno

Cada archivo recibe un **doc_id** con la forma **CARPETA-ARCH-FORMATO-N**, donde los dos primeros componentes son prefijos de cuatro caracteres del directorio y del nombre del archivo, y N es un contador global de recorrido. El contador global garantiza unicidad aunque dos archivos compartan prefijo. El **chunk_id** extiende el **doc_id** con el ordinal del fragmento dentro del documento.

El campo **fenomeno** se determina buscando los marcadores **F1_**, **F2_** y **F3_** en la ruta relativa completa del directorio, no en el nombre del archivo. Es la única señal fiable, ya que los nombres de archivo del corpus no codifican su temática. Los directorios de entorno virtual y control de versiones, así como las extensiones propias del proyecto (**.py**, **.faiss**, **.jsonl**), se excluyen del recorrido para que no contaminen el índice.

3. Estrategia de *chunking* y justificación

3.1. Por qué no se usó un segmentador genérico

La especificación establece un requisito obligatorio: ningún fragmento puede contener oraciones incompletas, y los cortes deben caer únicamente en límites oracionales. Los segmentadores recursivos por caracteres no pueden garantizar esa propiedad. Su mecanismo consiste en probar una lista ordenada de separadores y, cuando ninguno produce trozos suficientemente pequeños, recurrir al espacio en blanco y finalmente a la cadena vacía. En un corpus con párrafos largos —frecuentes en informes institucionales— ese respaldo se activa con regularidad y produce fragmentos cortados a mitad de frase, lo que constituye una violación directa del requisito.

La solución fue invertir el orden del razonamiento: en lugar de partir el texto y comprobar después dónde quedaron los cortes, se construyen los fragmentos *agregando oraciones completas* hasta agotar el presupuesto de tamaño. Bajo este esquema, la completitud lingüística no es una propiedad que haya que verificar, sino una consecuencia estructural del algoritmo.

3.2. Detección de fin de oración

El detector reconoce como frontera un signo terminal seguido opcionalmente de comillas o paréntesis de cierre y de espacio o salto de línea. Sobre esa base se aplican tres reglas de descarte, calibradas contra los patrones observados en el corpus:

- **Abreviaturas.** Un diccionario de unos setenta términos con punto que no cierran oración, cubriendo español, inglés y portugués: tratamientos (**sr**, **dr**, **ing**), referencias bibliográficas (**fig**, **vol**, **pág**, **cf**, **op**, **cit**), locuciones latinas (*i.e.*, *e.g.*, *etc*), siglas de país (**ee.uu**, **u.s.**, **u.k**) y grados militares (**gral**, **cnel**, **tte**, **cap**), estos últimos especialmente frecuentes en la documentación del Fenómeno 1.
- **Iniciales sueltas.** Una única letra seguida de punto corresponde a una inicial de nombre propio, no a un final de oración.
- **Numeraciones.** Un número seguido de punto se descarta como frontera únicamente si abre la línea, caso en que se trata de un ítem de lista enumerada. Si el número cierra la frase —un año, una cifra— sí constituye un final de oración legítimo. Esta distinción por posición evita tanto partir listas como fusionar párrafos completos.

3.3. Construcción de los fragmentos

El texto se divide primero en bloques por línea en blanco y, dentro de cada bloque, en oraciones. Las oraciones se acumulan en el fragmento actual mientras quepan simultáneamente en los dos presupuestos definidos (400 tokens y 240 palabras); cuando la siguiente oración no cabe, el fragmento se cierra y el siguiente arranca con la última oración del anterior. El solapamiento se expresa así en oraciones enteras, nunca en caracteres, con lo que una idea que queda a caballo entre dos chunks aparece completa en al menos uno de ellos.

Dos casos límite quedan cubiertos. Cuando una sola oración excede por sí misma los presupuestos —típico en filas de tablas, listas sin puntuación o salidas de OCR sin puntos— se activa un segmentador de emergencia que corta por separadores estructurales (salto de línea, barra vertical, punto y coma, dos puntos, coma) y solo en última instancia por espacios; este es el único punto del sistema donde puede aparecer un corte no oracional, y se restringe a texto que por su naturaleza carece de estructura oracional. En el extremo opuesto, los fragmentos de menos de quince tokens —títulos sueltos, colas de sección— se fusionan con su vecino siempre que el resultado no rebase los límites, evitando poblar el índice de vectores construidos sobre casi ningún contenido.

3.4. Parámetros y su justificación

Cuadro 3: Parámetros del segmentador y motivo de cada valor.

Parámetro	Valor	Justificación
max_tokens	400	El encoder admite 512 tokens. El margen de 112 absorbe el prefijo passage: y la variabilidad del tokenizador multilingüe, de modo que ningún fragmento se trunque silenciosamente.
max_palabras	240	El formato de salida limita cada fragmento a 250 palabras. Fijar 240 en la indexación hace que los chunks del índice <i>ya</i> cumplan el límite, y elimina la necesidad de subdividirlos al construir resultados.jsonl .
solape	1 oración	Aporta continuidad de contexto entre chunks vecinos con un coste de duplicación bajo y sin introducir cortes.
min_tokens	15	Umbral por debajo del cual un fragmento se considera demasiado pobre para producir un vector informativo.

El conteo de tokens se realiza con el tokenizador exacto del encoder, no con una aproximación por palabras, de forma que el presupuesto refleje el límite real del modelo. Como el segmentador consulta el tamaño de las mismas oraciones muchas veces —al evaluar el solape, al recortarlo y al fusionar— la tokenización está memorizada en caché, lo que reduce sustancialmente el tiempo de indexación sin alterar el resultado.

Cuadro 4: Distribución de longitud de los fragmentos.

	mín.	p25	mediana	p75	máx.
Tokens por fragmento	TODO	TODO	TODO	TODO	TODO
Palabras por fragmento	TODO	TODO	TODO	TODO	TODO

3.5. Metadata por fragmento

Cada fragmento registra los ocho campos obligatorios de la especificación, más un campo adicional **fecha** con la marca temporal del archivo de origen, disponible para post-filtros por rango de fechas. El campo **num_tokens** se calcula sobre el texto del fragmento sin el prefijo **passage:**, ya que el prefijo es un artefacto de codificación y no forma parte del contenido.

Listing 1: Ejemplo de línea de **metadata.jsonl**.

```
{"doc_id": "F1_I-INFO-PDF-37", "chunk_id": "F1_I-INFO-PDF-37-Chunk-12",  
  "fuente": "./F1_IA_y_Capacidades_Estrategicas/informe.pdf", "formato": "pdf",  
  "fenomeno": 1, "posicion": 12, "num_tokens": 187,  
  "fecha": "...", "texto": "..."}

```

4. Encoder seleccionado

4.1. Modelo y criterios de elección

Se seleccionó **intfloat/multilingual-e5-small**. La tabla siguiente contrasta el modelo con los seis criterios que plantea la especificación.

Cuadro 5: Evaluación del encoder frente a los criterios del reto.

Criterio	Justificación
Soporte multilingüe	El modelo se entrenó sobre un corpus multilingüe y sitúa español, inglés y portugués en un espacio compartido.
Dimensionalidad	384 dimensiones. Suficiente para recuperación densa y notablemente más económica en memoria y tiempo de búsqueda que las 768 o 1024 de los modelos mayores.
Longitud máxima	512 tokens, coherente con el presupuesto de 400 tokens fijado en el segmentador.
Rendimiento	La familia E5 está optimizada explícitamente para recuperación densa mediante entrenamiento contrastivo con pares consulta-pasaje, que es exactamente la tarea evaluada, y no para clasificación o similitud de pares.
Licencia	MIT
Eficiencia	La variante <i>small</i> permite indexar el corpus completo en CPU en un tiempo razonable, condición necesaria dado el volumen y la heterogeneidad del material.

4.2. Prefijos asimétricos de la familia E5

Los modelos E5 se entrenan con instrucciones asimétricas: los documentos se codifican con el prefijo **passage:** y las consultas con **query:**. La asimetría no es cosmética; el modelo aprende representaciones distintas para el rol de pregunta y el de respuesta, y omitir los prefijos degrada la recuperación de forma apreciable. La indexación aplica **passage:** a todos los fragmentos sin excepción, y el módulo de recuperación aplica **query:** a las 50 consultas.

4.3. Normalización de los vectores

Los embeddings se generan con `normalize_embeddings=True`, de modo que $\|v\| = 1$ y el producto interno coincide con la similitud coseno:

$$\cos(q, v) = \frac{q \cdot v}{\|q\| \|v\|} = q \cdot v. \quad (1)$$

Esta identidad es la que permite usar un índice de producto interno para obtener directamente ranking por coseno.

4.4. Sobre el uso de un único encoder

Se optó por un solo encoder en lugar de una combinación de varios. La fusión de rankings mediante CombSUM o RRF aporta robustez cuando los encoders tienen fortalezas complementarias, pero introduce un segundo índice, duplica el tiempo de codificación y añade una capa de hiperparámetros que no se puede calibrar sin un conjunto de validación con juicios de relevancia, del que no se dispone durante el reto. Ante esa imposibilidad de calibración, se prefirió invertir el esfuerzo en la calidad de la extracción y de la fragmentación, que afectan a todo el sistema aguas abajo.

5. Índice FAISS y almacén de metadata

5.1. Tipo de índice

Se emplea `IndexFlatIP` sobre vectores normalizados. Es un índice plano que compara la consulta contra todos los vectores y devuelve el resultado exacto. La elección responde a que la métrica de evaluación es NDCG@10: los índices aproximados como IVF o HNSW intercambian exactitud por velocidad, y cualquier vecino que dejen fuera se traduce directamente en pérdida de puntuación. Para el volumen del corpus, el coste de la búsqueda exhaustiva es irrelevante frente a las 50 consultas del conjunto de evaluación, de modo que no hay compensación que justifique aceptar recuperación aproximada.

5.2. Alineación índice–metadata

FAISS solo almacena vectores e identificadores enteros consecutivos asignados en el orden de inserción. La correspondencia con la metadata es, por tanto, puramente posicional: la línea n de `metadata.jsonl` describe el vector con identificador interno n . El sistema mantiene esta invariante construyendo, para cada archivo, dos listas paralelas —los textos con prefijo que se van a codificar y sus registros de metadata— que se emiten en el mismo orden y en el mismo punto del recorrido.

La verificación se realiza al cierre del proceso comprobando que `indice.ntotal` coincida exactamente con el número de líneas escritas. Cualquier discrepancia invalida la base, ya que desplazaría todos los resultados a partir del punto de desalineación.

5.3. Persistencia

```
base_vectorial/  
encoder_multilingual-e5-small/  
  index.faiss      # faiss.write_index  
  metadata.jsonl   # un objeto JSON por linea, orden = id interno FAISS
```

El índice se serializa con `faiss.write_index()` y es cargable con `faiss.read_index()` sin dependencias adicionales. La metadata se escribe en JSON Lines con `ensure_ascii=False` para preservar los caracteres acentuados del corpus en español y portugués.

6. Módulo de recuperación

6.1. Flujo de consulta

El script `generador.py` carga el índice y la metadata, y para cada una de las 50 consultas ejecuta la siguiente secuencia:

1. Se antepone el prefijo `query:` al texto de la consulta.
2. Se codifica con el mismo modelo y la misma normalización usados en la indexación, condición imprescindible para que el vector de consulta habite el mismo espacio semántico que el índice.
3. Se recuperan los k fragmentos de mayor producto interno.
4. Se leen los registros de metadata correspondientes por posición.
5. Se construyen los dos niveles de respuesta exigidos.

6.2. Agregación a nivel de documento

Los fragmentos recuperados se agrupan por `doc_id` y cada documento recibe como puntuación la de su fragmento más relevante (*max pooling*). Se prefirió esta estrategia a la suma de puntuaciones porque la suma favorece sistemáticamente a los documentos largos, que aportan más fragmentos al top- k por el simple hecho de tener más chunks en el índice. Dado que el corpus mezcla informes extensos con artículos breves, y que F1@3 es una métrica de conjunto sobre solo tres documentos, ese sesgo de longitud resultaría especialmente costoso. Los tres documentos de mayor puntuación agregada conforman la respuesta.

6.3. Cumplimiento del límite de 250 palabras

Los fragmentos del índice se construyeron con un techo de 240 palabras, por debajo del límite de 250 que impone el formato de salida. En consecuencia, ningún fragmento recuperado requiere subdivisión y el texto reportado coincide con el texto indexado, lo que preserva la correspondencia entre lo que se juzga y lo que efectivamente se recuperó.

6.4. Ausencia de modelos generativos

Ninguna etapa del sistema utiliza arquitecturas decoder. No hay reordenamiento mediante LLM, ni expansión o reformulación de consultas, ni filtrado basado en generación de texto, ni síntesis de fragmentos. La recuperación opera exclusivamente sobre vectores, puntuaciones de producto interno y campos de metadata. Los únicos modelos empleados son el encoder bidireccional E5 y el motor de OCR Tesseract, este último restringido a la fase de extracción de texto y sin intervención alguna en la recuperación.

7. Grafo de conocimiento

El componente bonus no se implementó. La extracción de relaciones sobre un corpus trilingüe y de dominio especializado exige un modelo de NER y un esquema de relaciones calibrados para el vocabulario del reto —organismos multilaterales, programas espaciales, entidades territoriales—, y un grafo construido con modelos genéricos habría producido tripletas ruidosas cuya fusión con el ranking vectorial arriesgaba degradar las métricas en lugar de mejorarlas. Se priorizó la solidez de la base vectorial, que es el componente obligatorio y el que determina ambas métricas de evaluación.

8. Reproducibilidad

El script se ejecuta desde la raíz del directorio de entrega:

```
python generador.py --consultas consultas.jsonl \
--base-vectorial ./base_vectorial \
--salida resultados.jsonl
```

Los tres argumentos son opcionales y sus valores por defecto reproducen la estructura de la entrega, de modo que la invocación `python generador.py` sin argumentos es equivalente al comando anterior.

Cuadro 6: Entorno de ejecución y dependencias principales.

Componente	Versión
Python	TODO
faiss-cpu	TODO
sentence-transformers	TODO
transformers	TODO
torch	TODO

La generación de `resultados.jsonl` a partir del índice ya construido toma aproximadamente TODO. El proceso es determinista: mismo índice y mismas consultas producen el mismo archivo de salida.

9. Limitaciones y trabajo futuro

- **Calidad del OCR.** Las páginas escaneadas y las infografías dependen de Tesseract, cuya salida carece con frecuencia de puntuación fiable. En esos casos el segmentador oracional no encuentra fronteras y recurre al mecanismo de emergencia, con fragmentos de menor calidad lingüística.
- **Cobertura del formato PBF.** Los mapas se convierten a texto atributo-valor, una representación pobre para consultas de naturaleza geográfica. Los atributos repetidos entre niveles de zoom pueden generar fragmentos redundantes.
- **Ausencia de deduplicación.** No se detectan documentos duplicados o casi idénticos dentro del corpus. Un mismo contenido replicado en varios archivos puede ocupar varias posiciones del top-10 y penalizar el NDCG por falta de diversidad. Un filtro por hash del texto normalizado, o un tope de fragmentos por documento en la lista final, son las extensiones naturales.
- **Un solo encoder.** La fusión de rankings mediante RRF con un segundo encoder es la vía más directa de mejora, pero requiere un conjunto de validación con juicios de relevancia para calibrarse con criterio.
- **Sin reordenamiento.** La prohibición de modelos generativos descarta el reranking habitual en sistemas de recuperación; un *cross-encoder* bidireccional sería compatible con las reglas y podría refinar el orden dentro del top-*k*.